

Ammerli Platform

Technical Documentation

B2B E-Commerce Platform
Connecting Wholesalers with Supermarkets

Version 1.0 - Staging

November 30, 2025

Contents

Document History **5**

1 Executive Summary **6**
1.1 Platform Overview . . . 6
1.2 Key Features . . . 6
1.3 Technology Stack Summary . . . 6

2 System Architecture **7**
2.1 Architecture Overview . . . 7
2.2 Component Breakdown . . . 8
2.2.1 Frontend Layer . . . 8
2.2.2 API Gateway Layer . . . 8
2.2.3 Application Layer . . . 9
2.2.4 Data Layer . . . 9
2.2.5 External Services . . . 10
2.3 Infrastructure . . . 10
2.3.1 Hosting Environment . . . 10

3 API Architecture **11**
3.1 API Design Principles . . . 11
3.2 API Versioning . . . 11
3.3 API Structure . . . 11
3.4 Authentication & Authorization . . . 11
3.4.1 JWT Token Structure . . . 11
3.4.2 Authentication Flow . . . 12
3.4.3 Authentication Flow . . . 12
3.4.4 Authorization Headers . . . 12
3.5 Role-Based Access Control (RBAC) . . . 13
3.5.1 User Roles . . . 13
3.6 API Modules Overview . . . 13
3.7 Common API Patterns . . . 14
3.7.1 Standard Response Format . . . 14
3.7.2 Pagination . . . 14
3.7.3 Filtering . . . 14
3.8 HTTP Status Codes . . . 14

4 Database Architecture **15**
4.1 Database Overview . . . 15
4.2 Entity Relationship Diagram (ERD) . . . 15
4.3 Database Connection Management . . . 16
4.3.1 PgBouncer Configuration . . . 16
4.4 Database Migrations . . . 16
4.5 Data Integrity . . . 16

5 Module Architecture **17**
5.1 Modular Structure . . . 17
5.2 Project Directory Structure . . . 17
5.3 Core Modules . . . 19
5.3.1 Authentication Module . . . 19
5.3.2 User Module . . . 19
5.3.3 Product Module . . . 20
5.3.4 Order Module . . . 20
5.3.5 Checkout Module (Chargily) . . . 21
5.3.6 Invoice Module . . . 21
5.3.7 Notifications Module . . . 22
5.3.8 Analytics Module . . . 23

6 Storage Architecture **24**
6.1 MinIO Object Storage . . . 24
6.1.1 Overview . . . 24
6.1.2 Bucket Structure . . . 24
6.1.3 Access Control . . . 24
6.1.4 File Size Limits . . . 25
6.2 Redis Caching Strategy . . . 25
6.2.1 Cache Usage . . . 25

7 External Integrations **25**
7.1 Chargily Payment Gateway . . . 25
7.1.1 Overview . . . 25
7.1.2 Integration Flow . . . 26
7.1.3 Configuration . . . 26
7.1.4 Webhook Handling . . . 26
7.2 SmsSak SMS Provider . . . 27
7.2.1 Overview . . . 27
7.2.2 Use Cases . . . 27
7.2.3 Configuration . . . 27
7.3 Firebase Cloud Messaging (FCM) . . . 27

7.3.1 Overview . . . 27
7.3.2 Setup . . . 27
7.3.3 Device Token Management . . . 28
7.3.4 Notification Types . . . 28

8 Security Architecture 29
8.1 Authentication Security . . . 29
8.1.1 Password Security . . . 29
8.1.2 JWT Token Security . . . 29
8.2 Authorization Security . . . 30
8.2.1 Role-Based Access Control . . . 30
8.2.2 Resource Ownership . . . 30
8.3 API Security . . . 30
8.3.1 Input Validation . . . 30
8.3.2 Rate Limiting . . . 30
8.3.3 CORS Configuration . . . 31
8.4 Data Security . . . 31
8.4.1 Data in Transit . . . 31
8.4.2 Data at Rest . . . 31
8.4.3 Webhook Security . . . 31
8.4.4 PCI DSS Compliance . . . 31
8.4.5 Algerian Data Protection Laws . . . 31
8.4.6 Security Best Practices . . . 31

9 Deployment Architecture 33
9.1 Infrastructure Overview . . . 33
9.1.1 Hosting Environment . . . 33
9.2 Docker Compose Architecture . . . 33
9.2.1 Container Services . . . 33
9.3 Backup Strategy . . . 33
9.3.1 PostgreSQL Backups . . . 33
9.4 CI/CD Pipeline . . . 34
9.4.1 GitHub Actions Workflow . . . 34
9.5 Monitoring & Logging . . . 34
9.5.1 Grafana Dashboards . . . 34
9.5.2 Loki Log Aggregation . . . 34
9.5.3 Alerting . . . 35

10 Code Structure & Conventions 36
10.1 Project Organization . . . 36
10.1.1 Root Directory . . . 36

10.1.2 TypeScript Conventions 36
10.1.3 NestJS Architectural Patterns 36
10.2 Testing Strategy (Future Implementation) 36

Document History

Version	Date	Changes	Author
1.0	November 30, 2025	Initial documentation release	Zakaria Goumri

Executive Summary

Platform Overview

Ammerli is a B2B e-commerce platform designed to digitally connect wholesalers with supermarkets in Algeria. The platform facilitates online business transactions, allowing wholesalers to list and sell their products while supermarkets can browse, order, and manage their purchases efficiently.

Key Features

0. User authentication and role-based access control (RBAC)
0. Product catalog management with search capabilities
0. Order placement and tracking workflow
0. Integrated payment processing via Chargily
0. Automated invoice generation (PDF)
0. Real-time push notifications via Firebase
0. Analytics dashboard for business insights
0. SMS notifications via SmsSak

Technology Stack Summary

Backend Framework:	NestJS (Node.js)
Mobile App:	Flutter
Database:	PostgreSQL with PgBouncer
Caching:	Redis
API Gateway:	Traefik
Object Storage:	MinIO (Self-hosted)
Monitoring:	Grafana + Loki
Deployment:	Docker Compose
CI/CD:	GitHub Actions
Search Engine:	Elasticsearch

System Architecture

Architecture Overview

Ammerli follows a **modular monolithic architecture**, where the application is organized into distinct modules with clear boundaries, but deployed as a single unit. This approach provides:

- Clear separation of concerns
- Easier development and debugging

This section details the primary components shown in the System Overview Diagram

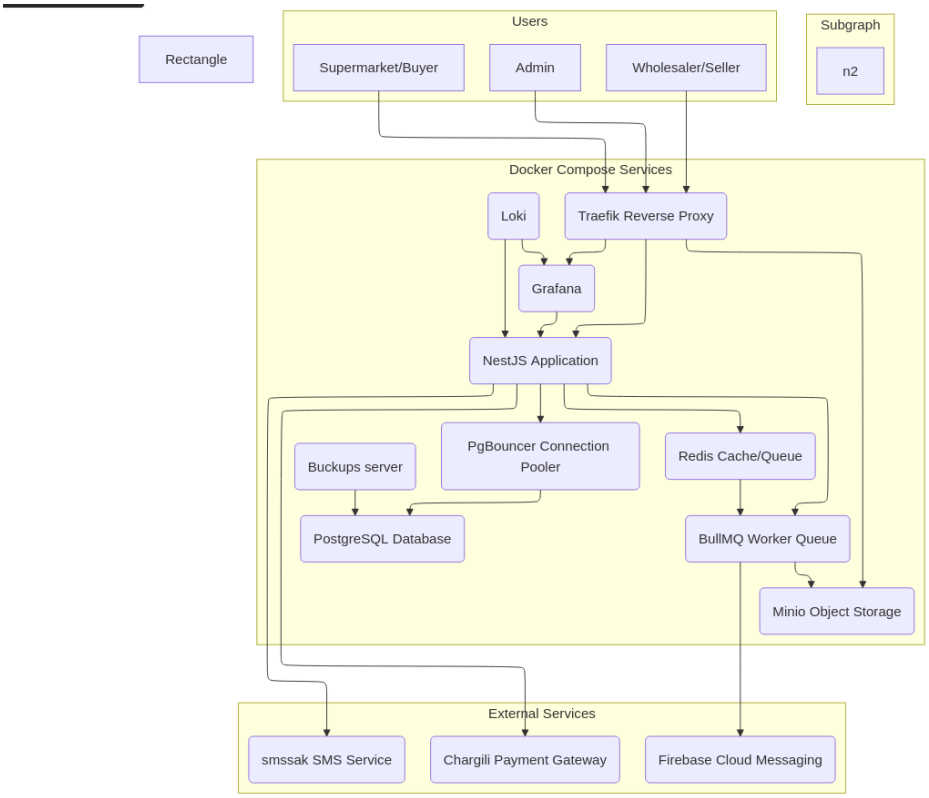


Figure 1: Overall System Overview of the Ammerli Platform

- **Flutter Mobile App:** The cross-platform client application serving both Supermarket (Buyer) and Wholesaler (Seller) (in v1 we only handled android version) users.

- **Traefik API Gateway:** Acts as the entry point for all external traffic, handling request routing, load balancing, and SSL/TLS termination.
- **NestJS Backend Application:** The core application, implemented as a modular monolith in TypeScript, housing all business logic and API endpoints, uses Fastify adapter.
- **PostgreSQL Database (with PgBouncer):** The primary relational database for persistent data storage, utilizing PgBouncer for efficient connection pooling.
- **Redis Cache:** Employed for high-speed data caching, session management, and as a message broker for background tasks (e.g., BullMQ queue).
- **MinIO Object Storage:** A self-hosted S3-compatible service for storing static assets such as product images, user profiles, and generated invoices.
- **External Services:** Integrations with third-party providers including:
  - Chargily (Payment Gateway) for online transactions.
  - SmsSak (SMS Service) for OTPs and notifications.
  - Firebase Cloud Messaging (FCM) for push notifications to mobile devices.
- **Grafana + Loki Monitoring Stack:** Provides real-time dashboards for system metrics and centralized log aggregation for operational insights.

Component Breakdown

Frontend Layer

- **Mobile Application:** Built with Flutter framework
- **Communication:** RESTful API calls to backend
- **Authentication:** JWT token-based (stored securely)
- **Notifications:** Firebase Cloud Messaging integration

API Gateway Layer

- **Technology:** Traefik reverse proxy
- **Responsibilities:**

  - Request routing
  - SSL/TLS termination
  - Load balancing (scaled horizontally 2 instances)
  - Rate limiting

Application Layer

- **Framework:** NestJS (TypeScript/Node.js)
- **Architecture Pattern:** Modular monolith
- **Core Modules:**
  - Authentication & Authorization
  - User Management
  - Product Management
  - Order Management
  - Payment Processing
  - Invoice Generation
  - Notifications
  - Analytics

Data Layer

Primary Database: PostgreSQL

- Stores all transactional data
- Connection pooling via PgBouncer
- Regular backups to backup server

Cache Layer: Redis

- Session management
- Frequently accessed data caching
- Rate limiting counters
- Temporary data storage

Object Storage: MinIO

- Self-hosted S3-compatible storage
- Stores images and files
- Presigned URLs for secure access
- Four main buckets: user-profile, products, categories, brand-pictures, invoices

External Services

Payment Gateway:	Chargily (Algerian payment processor)
SMS Provider:	SmsSak
Push Notifications:	Firebase Cloud Messaging (FCM)

Infrastructure

Hosting Environment

- **Provider:** Octinum VPS
- **Specifications:**
  - 160 GB SSD Storage
  - 4 CPU Cores
  - 8 GB RAM
- **Deployment:** Docker Compose orchestration

API Architecture

API Design Principles

Aammerli follows RESTful API design principles with:

- Resource-based URLs
- HTTP methods for CRUD operations (GET, POST, PUT, PATCH, DELETE)
- JSON request/response format
- Stateless communication
- Proper HTTP status codes

API Versioning

Versioning Strategy: URI-based versioning

```
app.enableVersioning({
  type: VersioningType.URI,
  defaultVersion: '1',
  prefix: 'api/v',
});
```

Example URLs:

- `https://api.3ammerli.com/api/v1/products`
- `https://api.3ammerli.com/api/v1/orders`

API Structure

All API endpoints follow this pattern:

`https://[domain]/api/v[version]/[resource]/[action]`

Authentication & Authorization

JWT Token Structure

- **Algorithm:** HS256 (HMAC with SHA-256)
- **Access Token:** 15 minutes expiration

- **Refresh Token:** 7 days expiration
- **Keys:** Separate signing keys stored in environment variables

Authentication Flow

Authentication Flow

The diagram below illustrates the comprehensive token-based authentication and refresh mechanism used by the platform to manage user sessions securely.

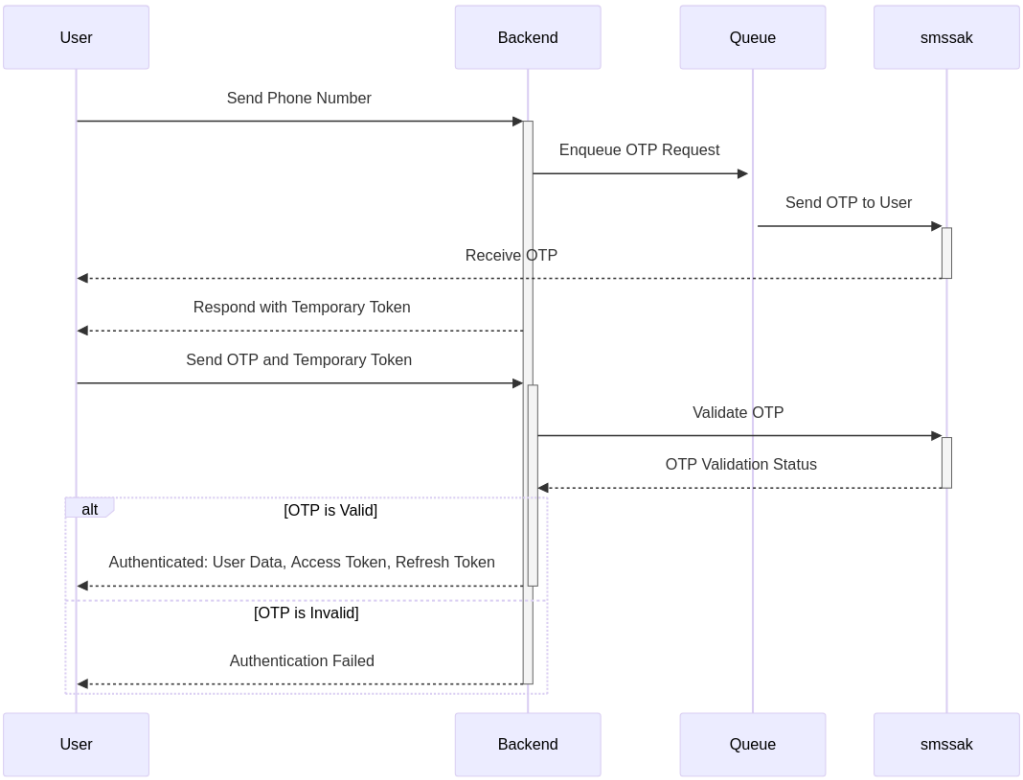


Figure 2: User Authentication, Token Generation, and Refresh Flow

Authorization Headers

All authenticated requests must include:

```
Authorization: Bearer <access_token>
```

Role-Based Access Control (RBAC)

User Roles

The system defines three primary user roles:

Role	Description
`ADMIN`	Platform administrator with full system access, including analytics dashboard and user management
`WHOLESELLER`	Sellers who manage product catalogs, inventory, pricing, and process orders
`SUPERMARKET`	Buyers who browse products, place orders, and manage their purchases

[Add specific permissions for each role, e.g.:]

- ADMIN: Full CRUD on all resources, access to analytics
- WHOLESELLER: CRUD on own products, view/update orders placed with them (not implimented yet)
- SUPERMARKET: Browse products, create orders, view own orders

API Modules Overview

Module	Endpoints
Authentication	`/api/v1/authentication/*` - otp/send, logout, refresh, register
Users	`/api/v1/users/*` - User profile, address management
Products	`/api/v1/products/*` - Product CRUD, search, categories, brands
Orders	`/api/v1/orders/*` - Order creation, tracking, status updates
Invoices	`/api/v1/invoices/*` - Invoice generation and retrieval
Notifications	`/api/v1/notifications/*` - Notification preferences, history
Analytics	`/api/v1/analytics/*` - Business metrics and reports
Category	`/api/v1/categories/*` - Category CRUD (Admin), list/details (All Users)
Brand	`/api/v1/brands/*` - Brand CRUD (Admin), list/details (All Users)
Health	`/health` - Service status check, uptime, dependency checks

Common API Patterns

Standard Response Format

Success response:

```
{
  "success": true,
  ...
}
```

Error response:

```
{
  "success": false,
  "error": {
    "message": "Human readable message translated depending on client langu
  }
}
```

Pagination

Query parameters for list endpoints:

GET /api/v1/products?page=1&limit=20&sort=createdAt:DESC

Response with pagination metadata:

```
{
  "data": [...],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
    "hasMore:true
  }
}
```

Filtering

HTTP Status Codes

Code	Usage
200	Successful GET, PUT, PATCH requests
201	Successful POST (resource created)
204	Successful DELETE (no content)
400	Bad request (validation error)
401	Unauthorized (missing/invalid token)
403	Forbidden (insufficient permissions)
404	Resource not found
409	Conflict (duplicate resource)
422	Unprocessable entity (semantic errors)
500	Internal server error

Database Architecture

Database Overview

- **Database Management System:** PostgreSQL
- **Connection Pooler:** PgBouncer
- **Migration Tool:** TypeORM Migrations

Entity Relationship Diagram (ERD)

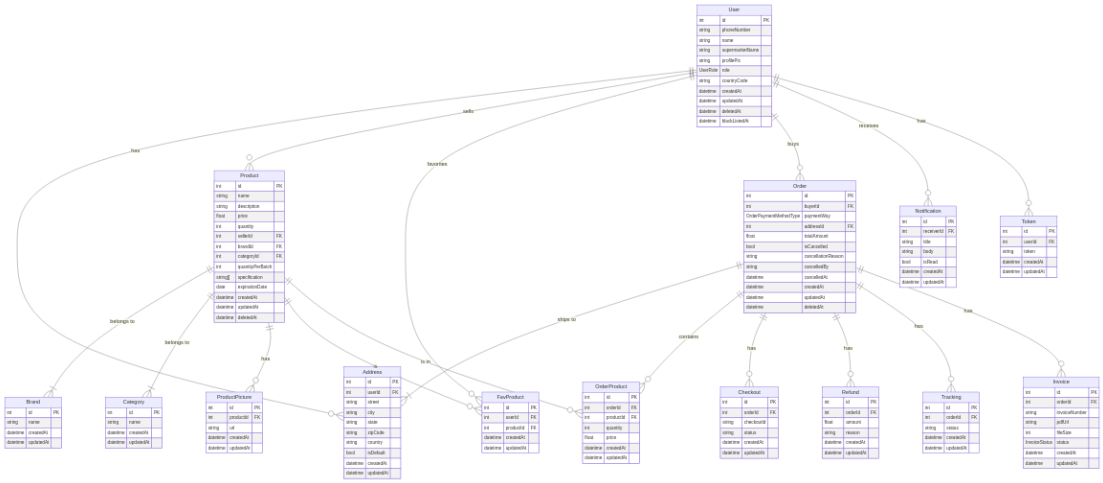


Figure 3: Comprehensive Entity-Relationship Diagram (ERD) for Ammerli Platform

Database Connection Management

PgBouncer Configuration

PgBouncer is used as a connection pooler to:

- Reduce connection overhead to PostgreSQL
- Handle connection pooling efficiently
- Manage connection limits
- Improve performance under load

Database Migrations

- **Tool:** TypeORM Migrations (NestJS default)
- **Location:** `src/migrations/`
- **Naming Convention:** `TIMESTAMP-MigrationName.ts`

Migration Workflow:

0. Generate migration: `pnpm run migration:generate - MigrationName`
0. Review generated SQL
0. Run migration: `pnpm run migration:run`
0. Revert if needed: `pnpm run migration:revert`

Data Integrity

- Foreign key constraints enforced
- Check constraints on enums (status fields)
- NOT NULL constraints on required fields
- Unique constraints on emails, phone numbers, order numbers, invoice numbers

Module Architecture

Modular Structure

The application follows a modular monolithic architecture, organized under `src/core/`. Each module is self-contained with its own:

- Controllers (HTTP endpoints)
- Services (business logic)
- Entities (database models)
- DTOs (Data Transfer Objects for validation)
- entites (database tables)

Project Directory Structure

Directory Tree Diagram: Insert a visual tree showing:

```
src/
 app.controller.ts
 app.module.ts
 app.service.ts
 data-source.ts
 main.ts
 common/
    constants/
    decorators/
    dtos/
    entities/
    enums/
    filters/
    interceptors/
    modules/
    scripts/
    types/
    utils/
 config/
    app.config.ts
    auth.config.ts
    chargili.config.ts
```

```
    cloud.config.ts
    db.config.ts
    firebase.config.ts
    minio.config.ts
    redis.config.ts
    sms.config.ts
    throttler.config.ts
    interfaces/
core/
    core.module.ts
    analytics/
    authentication/
    chargili/
    invoice/
    notifications/
    order/
    product/
    user/
i18n/
    ar/
    en/
    fr/
infrastructure/
    infrastructure.module.ts
    clustring/
    db/
    firebase/
    minio/
    object-storage/
    queue/
migrations/
monitoring/
    monitoring.module.ts
    health/
    logger/
security/
    security.module.ts
    rate-limiting/
seed/
     seed-runner.ts
```

```
seed.module.ts
seed.service.ts
seeds-data.ts
```

Core Modules

Authentication Module

Location: `src/core/authentication/`

Responsibilities:

- User registration (wholesaler and supermarket signup)
- Login with otp/phone
- JWT token generation (access & refresh)
- Token refresh mechanism
- Logout

Security Implementation:

- JWT tokens signed with HS256 algorithm
- Separate signing keys for access and refresh tokens
- Token expiry: 15 minutes (access), 7 days (refresh)
- Blacklisted users

User Module

Location: `src/core/user/`

Responsibilities:

- User profile management
- Address management (delivery/billing)
- User settings and preferences
- Profile picture upload (to MinIO)
- Account status management

Product Module

Location: `src/core/product/`

Responsibilities:

- Product CRUD operations (wholesalers only)
- Product catalog browsing (all roles)
- Category management
- Brand management
- Product image upload to MinIO
- Favorite products (wishlist)
- Product search via Elasticsearch
- Stock management

Search Implementation:

- Like DB querying integration for full-text search and optimize fees

Order Module

Location: `src/core/order/`

Responsibilities:

- Order creation by supermarkets
- Order status management by wholesalers
- Order tracking history
- Order cancellation
- Order listing with filters

Order Status Workflow:

0. **PENDING** - Initial state after order creation
0. **CONFIRMED** - Wholesaler confirms the order
0. **PREPARATION** - Wholesaler preparing the order
0. **ON_DELIVERING** - Order in transit
0. **DELIVERED** - Order successfully delivered
0. **CANCELLED** - Order cancelled (by either party)

Checkout Module (Chargily)

Location: `src/core/order/checkout`

Responsibilities:

- Chargily payment gateway integration
- Payment link generation
- Webhook handling for payment status updates
- Payment verification
- Transaction record keeping

Payment Flow:

0. Supermarket creates an order
0. System generates Chargily payment link
0. User redirected to Chargily for payment
0. Chargily sends webhook notification on payment completion
0. System updates order payment status
0. Invoice generated automatically
0. Push notification sent to user

Invoice Module

Location: `src/core/invoice/`

Responsibilities:

- Automated PDF invoice generation
- Invoice storage in MinIO
- Invoice retrieval and download
- Invoice numbering system

Invoice Generation Trigger: Invoices are automatically generated when:

- Order payment is confirmed (via Chargily webhook)

- Order status changes to DELIVERED (optional)

Invoice Content:

- Invoice number and date
- Wholesaler information
- Supermarket information
- Order items with quantities and prices
- Subtotal, tax, delivery fee, total
- Payment status and method

Notifications Module

Location: `src/core/notifications/`
Responsibilities:

- Push notifications via Firebase Cloud Messaging (FCM)
- SMS notifications via SmsSak
- Notification history
- Device token management
- Notification preferences

Notification Triggers:

- Order status changes
- Payment status updates
- Product expiration alerts
- New product announcements
- Promotional messages
- Product expiration
- Product low quantity

Analytics Module

Location: `src/core/analytics/`

Responsibilities:

- Business metrics calculation
- Sales turnover reporting
- Top-selling products analysis
- Dashboard data aggregation
- Admin-only access

Database Queries: Uses SQL Typeorm query builder to optimize querying

Storage Architecture

MinIO Object Storage

Overview

Ammerli uses MinIO, a self-hosted S3-compatible object storage system, for storing files and images.

Advantages:

- Full data ownership and control
- Cost-effective for high storage needs
- S3-compatible API
- High performance
- Easy integration with AWS SDKs

Bucket Structure

The system uses four dedicated buckets:

Bucket Name	Purpose
`user-profile`	User profile pictures and avatars
`products`	Product images (multiple per product)
`categories`	Category banner and icon images
`brand-pictures`	Brand logos and images
`invoices`	Generated PDF invoices

Access Control

Presigned URLs:

- All file access uses presigned URLs
- URLs are temporary and time-limited
- Provides secure access without exposing credentials
- Generated on-demand by backend

Upload Flow:

0. Client requests upload permission from backend
0. Backend generates presigned upload URL (PUT)
0. Client uploads file directly to MinIO
0. Backend receives confirmation and stores URL in database

Download Flow:

0. Client requests file from backend
0. Backend generates presigned download URL (GET)
0. Client downloads file directly from MinIO

File Size Limits

- User profile images: Max 5 MB
- Product images: Max 10 MB
- Multiple images per product: Max 3 images

Redis Caching Strategy

Cache Usage

Redis is used for:

- Session data (optional, if using session-based auth alongside JWT)
- Frequently accessed product data
- Category and brand lists
- Rate limiting counters
- Temporary data (OTP codes, password reset tokens)
- Elasticsearch query result caching

External Integrations

Chargily Payment Gateway

Overview

Chargily is an Algerian payment processor supporting local payment methods.

Integration Flow

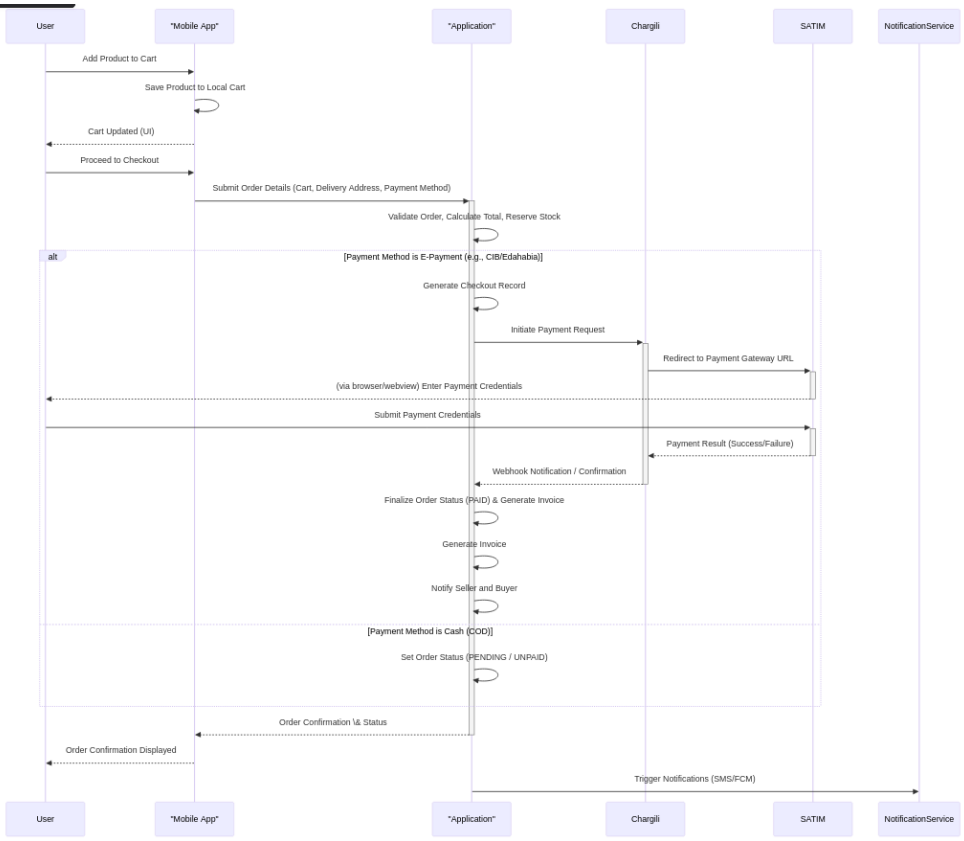


Figure 4: Order E-Payment Flow via Chargily and SATIM

Configuration

Environment variables required:

- `CHARGILY_API_KEY` - API authentication key
- `CHARGILY_SECRET_KEY` - Webhook signature verification
- `CHARGILY_WEBHOOK_URL` - Callback endpoint

Webhook Handling

Security:

- Verify webhook signature using secret key

- Validate payment amount matches order
- Prevent replay attacks (check transaction ID)
- Log all webhook events

Webhook Events:

- `payment.success` - Payment completed
- `payment.failed` - Payment failed
- `payment.cancelled` - User cancelled payment

SmsSak SMS Provider

Overview

SmsSak provides SMS messaging services for Algeria.

Use Cases

- OTP verification during registration

Configuration

Environment variables:

- `SMSSAK_API_KEY`
- `PROJECT_ID`

Firebase Cloud Messaging (FCM)

Overview

FCM handles push notifications to mobile devices (iOS and Android).

Setup

- Firebase project created
- Service account credentials stored in environment
- Flutter app configured with Firebase

Device Token Management

Token Registration:

- App requests FCM token on first launch
- Token sent to backend via API
- Stored in `notification_tokens` table
- Multiple devices per user supported

Token Lifecycle:

- Tokens can expire or be revoked
- App refreshes tokens periodically
- Backend removes invalid tokens after failed sends

Notification Types

Type	**Trigger**
Order Status	Order status changes (confirmed, delivered, etc.)
Payment Status	Payment success or failure
Product Alert	Product expiring soon (for wholesalers)
Promotional	Marketing messages (with user consent)
System	Important account or system notifications

Security Architecture

Authentication Security

Password Security

- Passwords hashed using bcrypt (10+ rounds)
- Never stored in plain text
- Minimum password requirements enforced
- Password reset via secure token (time-limited)

JWT Token Security

Access Tokens:

- Algorithm: HS256
- Expiry: 15 minutes
- Payload: user ID, role, minimal claims
- Signed with secret key from environment

Refresh Tokens:

- Algorithm: HS256
- Expiry: 7 days
- Different signing key than access tokens
- Used only for token refresh endpoint
- Invalidated on logout (blacklisted)

Token Transmission:

- Sent via `Authorization: Bearer` header
- Never in URL parameters
- HTTPS in production

Authorization Security

Role-Based Access Control

- Guards check user role before endpoint access
- Roles: ADMIN, WHOLESELLER, SUPERMARKET
- Role checked against endpoint requirements
- Unauthorized access returns 403 Forbidden

Resource Ownership

- Users can only access/modify their own resources
- Wholesalers can only manage their products
- Supermarkets can only view their orders
- Admins have override permissions

API Security

Input Validation

- All inputs validated using DTOs (class-validator)
- Type checking, length limits, format validation
- SQL injection prevention via parameterized queries (TypeORM)
- XSS prevention via input sanitization

Rate Limiting

- General API: 50 requests/minute per IP
- Bigger API: 100 requests/minute per IP
- Subnet Rate limting :150 req/minute per Ip
- otp endpoint: 1 attempt/30s per IP
- Implemented via Redis counters
- Returns 429 Too Many Requests

CORS Configuration

- Allowed origins: Flutter mobile app only

Data Security

Data in Transit

- HTTPS/TLS for all API communications
- SSL certificate configured on Traefik
- Certificate auto-renewal via Let's Encrypt

Data at Rest

- Database: PostgreSQL encryption
- File storage: MinIO access control private and use presigned urls

Webhook Security

- Chargily webhooks: signature verification
- Reject unsigned or invalid requests
- Log all webhook attempts

PCI DSS Compliance

Not directly applicable as payment card data is handled entirely by Chargily. However:

- No card data stored in system
- Secure transmission to payment processor
- Maintain secure network

Algerian Data Protection Laws

Security Best Practices

- Principle of least privilege
- Defense in depth approach

- Regular security audits
- Dependency vulnerability scanning
- Keep all packages updated
- Secure deployment practices

Deployment Architecture

Infrastructure Overview

Hosting Environment

Provider:	Octinum VPS
Storage:	160 GB SSD
CPU:	4 Cores
RAM:	8 GB
OS:	[Add OS, e.g., Ubuntu 22.04 LTS]

Docker Compose Architecture

Container Services

The application runs as multiple Docker containers orchestrated by Docker Compose:

Docker Compose :

- Traefik (API Gateway & Reverse Proxy)
- NestJS Backend Application
- PostgreSQL Database
- PgBouncer (Connection Pooler)
- Redis Cache
- MinIO Object Storage
- Grafana (Monitoring UI)
- Loki (Log Aggregation)

Backup Strategy

PostgreSQL Backups

- **Method:** pg_dump automated script
- **Frequency:** daily

- **Retention:** keep data of the last 4 days
- **Storage:** Separate backup server

CI/CD Pipeline

GitHub Actions Workflow

Trigger: Push to main branch
Pipeline Stages:

0. **Code Checkout**
0. **Build**
0. **Push Image**
0. **Deploy**
0. **Verify**
    - Health check endpoint verification
    - Rollback on failure

Monitoring & Logging

Grafana Dashboards

Grafana provides visualization for:

- Business Dashboard
- Logs Dashboard
- Error Dashboard
- Performance Dashboard

Loki Log Aggregation

Loki collects logs from:

- NestJS application logs
- Traefik access logs
- PostgreSQL logs

- Container stdout/stderr

Log Levels:

- ERROR: Critical errors requiring attention
- WARN: Warning messages
- INFO: General information (production)
- DEBUG: Detailed debugging (development)

Alerting

- High error rate alerts
- System resource alerts
- Database connection failures
- External service failures
- Notification channels: [Slack]

Code Structure & Conventions

Project Organization

Root Directory

TypeScript Conventions

All application code must adhere to the Airbnb TypeScript style guide, enforced by ESLint. Key conventions include:

- Use **PascalCase** for classes, interfaces, and types.
- Use **camelCase** for variables, functions, and properties.
- Explicitly define return types for functions and methods.

NestJS Architectural Patterns

- **Controllers:** Handle HTTP requests and delegate business logic to services. They should be lean, primarily dealing with request validation (via DTOs/Pipes) and response formatting.
- **Services:** Contain all core business logic, database interaction logic (via repositories), and integration with external APIs.
- **Modules:** Encapsulate related components (controllers, services, entities) and manage dependencies.

Testing Strategy (Future Implementation)

The testing strategy will follow NestJS best practices, covering:

- **Unit Tests:** For isolated services, pipes, and guards.
- **Integration Tests:** For controllers and service interactions, using mocked dependencies.
- **End-to-End (E2E) Tests:** For full API request lifecycle validation (using tools like Supertest).